Session 21 — Study Guide: Real Data Migration and the First Real Route Cutover

Status: PREPARED DRAFT

This guide is written for someone reviewing alone tonight, maybe thinking hard for the first time about what it actually takes to move real data between two completely different databases. If a sentence assumes you remember something from Session 19 or 20, it says so and reminds you what that something was.

What we learned today

Four real things, in order: (1) the real difference between a SCHEMA migration (what Sessions 19-20 did) and a DATA migration (mostly what today does), and the one real precondition a data migration has: the target schema must actually be able to represent the source, checked directly; (2) TWO real mismatches found by that check — a genuinely missing `Workspace.description` column, closed with one small, real, generated migration, and a genuine field-mapping problem (`Workspace.ownerId` isn't stored where you'd expect); (3) how to write a real, safe, re-runnable script that moves real `User` and `Workspace` data from MongoDB into Postgres, including both fixes; (4) how to cut over a real route from Mongoose to Prisma, bounded and reversible, while proving nothing else broke.

Why we needed it

Every earlier Prisma session worked against either an empty table or data this project itself seeded for teaching purposes. Real products do not get that luxury: the real data already exists, in a different database, and someone has to move it — correctly, safely, without losing anything — before any route can actually switch to reading it. That "someone" is the script this session builds.

Mental model

Think of the real Mongo data as boxes already packed and labeled, sitting in one warehouse. The Postgres tables are empty shelves in a SECOND warehouse, already built to the right shape (Session 19 did that). Today's script is the delivery truck: it reads each real box's label, decides where it goes on the new shelves, and — if a box's label is unreadable or missing information the new shelf requires — sets it aside with a note explaining why, instead of guessing or jamming it onto the wrong shelf.

First-Time Tool Box — one script, two real databases

Every earlier script in this project talked to exactly one database. Today's script is the first one to use the MongoDB driver AND Prisma Client together, in the same file.

What it is. `MongoClient` (from the `mongodb` package — the same real driver Mongoose itself uses underneath) opens a direct connection to MongoDB, independent of Nest/Mongoose entirely. `PrismaClient` is the same tool you already know from Sessions 19-20.

Why we use it here. A data migration script's whole job is to read from one real database and write to a different one — it genuinely needs both clients open at once, in the same script.

Where/how you run it. A plain standalone script, run from `api/` with `ts-node`, exactly like Session 20's own homework proof script — never through Nest's HTTP layer.

The commands you actually need today:

BASH

```
npx ts-node prisma/migrate-users-workspaces.ts   # run the real migration
npx ts-node prisma/verify-migration.ts           # run the real verification (homework)
psql -d b4f_board -c "SELECT ..."                # check Postgres directly
mongosh b4f-board --eval "db.<collection>.find()" # check MongoDB directly
```

How to tell it worked. The script prints real counts for what it found and what it did; the real proof is always querying BOTH databases directly afterward and comparing them yourself, not trusting the printed summary alone.

Common first-time mistakes: forgetting to await `mongo.connect()` before querying (every `db.collection(...)` call would fail); closing the Mongo connection (`mongo.close()`) before every real read finishes, instead of only at the very end; forgetting `import 'dotenv/config'` as the very first line — a real, empirical test confirms plain `ts-node` does not load `.env` on its own, unlike Nest's own bootstrap or Prisma's own CLI; both of which this script is neither.

The problem, before the solution

Three real things make this harder than any earlier migration:

1. **The two schemas don't store the same fields — and one gap is a missing COLUMN, not just a missing VALUE.** The real Mongoose `Workspace` document has `id`, `name`, `slug`, `description`. The real Prisma `Workspace` model (built in Session 19, before `Workspace.description` existed anywhere in this project's own real data) had no `description` column at ALL — not "stored differently," genuinely nowhere to put it. That gets closed first, with one small, real, generated migration, before any data moves.
2. **A second gap is a missing RELATIONSHIP, not a missing column.** The real Prisma `Workspace` model also requires `ownerId` — a field that does not exist on the Mongo `Workspace` document at all, but IS real data, stored somewhere else (see Step 1 below). This is a different kind of problem from #1: nothing in `schema.prisma` needs to change for this one.
3. **Some real data genuinely cannot be migrated safely.** A small number of real workspaces have no resolvable owner at all (see "The solution, step by step" below) — a script that ignores this and guesses an owner, or crashes outright, is a real script you cannot trust.

The solution, step by step

Step 0 — close the schema gap first, with a real, generated migration

PRISMA

```
model Workspace {
  id          Int      @id @default(autoincrement())
  name        String
```

```
  slug        String   @unique
  description String   @default("")
  ownerId     Int      @map("owner_id")
  createdAt   DateTime @default(now()) @map("created_at")
  // ...
}
```

BASH

```
npx prisma migrate dev --name add_workspace_description
```

What is it? A real, GENERATED schema migration — never hand-written SQL. **Why do we need it?** Because a data migration can only preserve data the TARGET schema can actually represent; if Postgres has nowhere to put `description`, the value is lost no matter how careful the rest of the script is. **What breaks without it?** Every real workspace's own description would be silently dropped during migration — exactly the kind of mistake a quick glance at "it printed PASS" would never catch. **What to look at:** the real generated `migration.sql` — `ALTER TABLE "workspaces" ADD COLUMN "description" TEXT NOT NULL DEFAULT '';`. **What to expect:** `@default("")` matches the real Mongoose default (`@Prop({ default: '' })`) exactly.

Step 1 — find out where ownership actually lives

Real ownership is not on the `Workspace` document — it's a separate `WorkspaceMember` document, with `role: 'owner'`, linking a `workspaceId` to a `userId`:

TS

```
const ownerMember = await db
  .collection('workspacemembers')
  .findOne({ workspaceId: w.id, role: 'owner' });
```

What is it? A real query against a DIFFERENT real collection than the one being migrated. **Why do we need it?** Because `ownerId` has to be DERIVED, never invented. **What breaks without it?** You'd have no real value at all for a required Postgres column. **What to look at:** the real Mongoose `WorkspaceMember` schema (`workspaceId`, `userId`, `role`). **What to expect:** one matching document for a properly-created workspace, `null` for one that has no owner at all.

Step 2 — migrate Users first, preserving real ids

TS

```
import 'dotenv/config';
// ... real client setup ...

await prisma.user.upsert({
  where: { id: u.id },
  update: {},
  create: {
    id: u.id,
    name: u.name,
    email: u.email,
```

```
    passwordHash: u.passwordHash,
    createdAt: u.createdAt ?? new Date(),
  },
});
```

`upsert()` (not `create()`) is what makes re-running the script SAFE FROM DUPLICATES — `update: {}` means "if this id already exists, change nothing; otherwise, create it." Be precise about what that actually guarantees: safe to re-run without duplicating, never "always keeps Postgres synchronized with Mongo" — if a migrated row's data in Postgres is later changed independently, re-running this script will NOT repair it. `id: u.id` preserves the SAME real identity the user already had — without it, Postgres's own autoincrement would hand out a DIFFERENT number, breaking any later foreign key that needs to point back at this exact user. Users must be migrated BEFORE Workspaces, because Workspace's own `ownerId` is a real foreign key into `User`. `import 'dotenv/config'` must be the very first line — before `@prisma/client` is even imported — because `PrismaClient`'s own constructor already needs `DATABASE_URL` to be real.

Step 3 — migrate Workspaces, skipping what can't be done safely

TS

```
if (!ownerMember) {
  skipped.push({
    id: w.id,
    name: w.name,
    reason: 'no WorkspaceMember with role=owner found',
  });
  continue;
}

const ownerExists = await prisma.user.findUnique({
  where: { id: ownerMember.userId },
});

if (!ownerExists) {
  skipped.push({
    id: w.id,
    name: w.name,
    reason: `owner userId ${ownerMember.userId} does not exist in migrated Users`,
  });
  continue;
}
```

Two real, independent failure modes, each caught and reported BEFORE Prisma ever sees a bad value: no owner reference exists at all; or the owner reference points at a user that was never actually migrated. Both `skipped.push({ id, name, reason })` calls record exactly what happened and move on — never a guess, never an uncaught crash.

Once a workspace passes both checks, it is migrated with `description: w.description ?? ''` alongside `name` / `slug` / `ownerId` — the real column Step 0 just added, now carrying real data.

One more real gap: Postgres's own autoincrement sequence

TS

```
await prisma.$executeRawUnsafe(
  `SELECT setval(pg_get_serial_sequence('users', 'id'), COALESCE((SELECT MAX(id) FROM
users), 1), true);`,
);
await prisma.$executeRawUnsafe(
  `SELECT setval(pg_get_serial_sequence('workspaces', 'id'), COALESCE((SELECT MAX(id) FROM
workspaces), 1), true);`,
);
```

What is it? Resetting Postgres's own `SERIAL` sequence to the real current max id, run at the end of every migration. **Why do we need it?** `id: u.id` / `id: w.id` inserts an EXPLICIT id — Postgres's own autoincrement sequence never learns about that on its own; it only advances on a DEFAULT-id insert. **What breaks without it?** The next real Prisma-native `create()` (no explicit id — a normal `POST /workspaces`) could try to use an id this script already migrated, and collide. **What to look at:** run it, then try a real `create()` and confirm the new id is higher than every migrated one. **What to expect:** this is itself safe to run every time — `setval` to the same value twice does nothing extra.

Why you verify BEFORE you cut over, never after

What is it? Running the real verification checks (real counts matching, every owner resolving, every slug unique) against the migrated Postgres data, before any real route is pointed at it. **Why do we need it?** A route cutover makes Postgres the thing real users actually see — if the migration silently lost or mismapped something, the cutover would be the moment that mistake becomes visible to them, not before. **What breaks without it?** You would be trusting data you never actually checked. **What to look at:** the real migration script's own printed counts (LC-05/LC-07), then a direct `psql` / `mongosh` comparison — verification is not optional homework, it is a precondition for the cutover that follows it. **What to expect:** by the time LC-10 begins, every real check has already passed for real, not merely been assumed.

Why the application can run on two databases at once, safely, for now

This is not a temporary bug to apologize for — it is the normal, expected shape of a real, in-progress migration. `Workspaces` 's three READ routes answer from Postgres; its own write routes, and every other resource in this project, still answer from MongoDB, at the exact same time, in the exact same running app. Nothing requires an all-or-nothing switch: each resource (or even each operation on a resource) can move independently, once its own real data has been migrated and verified — exactly the "resource-by-resource, never one giant rewrite" plan this project committed to back in Session 19.

Step 4 — the real cutover (inside the SERVICE, not the controller)

TS

```
// api/src/workspaces/workspaces.service.ts
async findOne(id: number) {
  const workspace = await this.prisma.workspace.findUnique({
```

```
    where: { id },
  });

  if (!workspace) {
    throw new NotFoundException(`No workspace found with id ${id}.`);
  }
  return workspace;
}
```

`findUnique` returns `null` on a miss — unlike the old Mongoose-backed version, which already threw `NotFoundException` internally. The explicit `throw` here is what keeps the real `404` contract identical for anyone already calling this route. **Where this lives matters.** `api/src/workspaces/workspaces.controller.ts` is never edited this session — its own `findOne()` already just called `this.workspacesService.findOne(id)`, and still does. The controller's job is HTTP concerns (reading the param, delegating, returning what it gets back); the service's job is deciding which database actually answers that call. Putting the Prisma call directly in the controller instead would work, but it would put a persistence decision in the one layer whose whole point is not needing to know about persistence.

What "prove old behavior still works" actually means

Not byte-identical JSON — a Mongoose document always carries `_id` / `__v`; a Prisma row never does. It means two different things, kept distinct: every field that existed BEFORE this cutover (`id` / `name` / `slug` / `description` — real, pre-existing Mongoose data) must still be there with the SAME real value, checked against the real Mongo source; `ownerId` / `createdAt` are NEW fields this migration introduces — there's nothing in the old response to regression-test them against, so the real proof for those two is that they resolve correctly, not that they "match the old response." A missing resource still returns a real `404` with the same real message. The exact shape changes; nothing that mattered is gone.

Connecting to earlier sessions

- **Session 19** built the Postgres `User` / `Workspace` tables this session migrates real data into. Today adds exactly one small new migration on top (`add_workspace_description`) — found necessary by actually checking the real shapes side by side, not assumed away.
- **Session 20** proved `{ increment: 1 }`'s own atomicity and built the `PrismaService` convention this session's cutover reuses. `PrismaModule` is `@Global()`, so `PrismaService` can be injected anywhere without re-importing `PrismaModule` — that is a dependency-injection convenience, not a decision about WHICH layer should own database access. This session injects it into `WorkspacesService`, never the controller.
- Today is the first time either Session's own real Mongo data (not seed/demo data invented for a teaching slice) is actually read and moved.

Glossary

- **Data migration** — moving/transforming real existing rows between databases; never changes table structure (contrast with a schema migration, Sessions 19-20's own kind).
- **`upsert()`** — "insert if missing, otherwise leave alone" (here); the real, idiomatic way to make a migration script safe to re-run.
- **Route cutover** — switching which real implementation answers an existing route, without changing the route's own real contract (same path, same meaningful response shape, same error behavior).
- **Field mapping** — deciding how a value in one schema corresponds to a (possibly differently-shaped, possibly missing) value in another.
- **Skip-and-report** — a real migration script's own discipline: when a row cannot be migrated safely, record exactly why and move on, never guess a fallback value, never crash uncaught.
- **Schema gap** — a value the source has that the target schema has nowhere to hold at all (a missing column), closed with a real, generated migration — different from a field-mapping problem, where the value exists in the target's reach, just under a different real shape.

Common beginner mistakes

- Migrating Workspaces before Users — the `ownerExists` check would then fail for everyone, since no user would exist yet to resolve against.
- Writing `prisma.workspace.create()` instead of `.upsert()` once a workspace already exists — throws a real unique-constraint error on a second run.
- Assuming every real workspace has a resolvable owner — a real, disclosed minority do not.

Common errors and how to debug them

- **A real `P2003` foreign-key violation on `owner_id`.** This means a workspace was migrated with an `ownerId` that doesn't actually exist in `User` — this should never happen if the `ownerExists` check is in place and correct; if you see this, re-check that the check actually runs BEFORE the `upsert()`, not after.
- **The cutover route returns `500` instead of `404` for a missing id.** Inspect whether the explicit `NotFoundException` throw is actually present — `findUnique` returning `null` silently, with no throw, would otherwise fall through as a `200` with a `null` body, not a clean `404`.
- **`MongoClient` connection hangs, or `MONGODB_URI` / `DATABASE_URL` reads as `undefined`.** Check that `import 'dotenv/config'` is actually the first line of the file — a real, empirical test confirms plain `ts-node` does not load `.env` on its own; never hardcode a connection string into the script itself.
- **The migration script reports fewer users migrated than you expect.** Never run `prisma migrate reset` to "start clean" and re-test — that destroys the real data this whole lesson depends on; inspect the real Mongo collection directly instead.

Review questions

1. What is the real difference between a schema migration and a data migration, and what real precondition does a data migration always have?

2. LC-03 finds TWO real mismatches. Name both, and explain why they are DIFFERENT kinds of problem, fixed in different places.
3. Name the two real, independent reasons a workspace might be skipped during migration.
4. What does `upsert()`'s own `update: {}` actually guarantee — and what does it NOT guarantee?
5. Why does a standalone script need `import 'dotenv/config'` when the Nest app itself doesn't need anything like it?
6. Why does Postgres's own autoincrement sequence need to be synced after this migration runs?
7. What does "prove old behavior still works" actually require, given that the JSON shape itself changes, and two brand-new fields (`ownerId` / `createdAt`) now exist?

Answers

1. A schema migration changes table structure; a data migration moves/transforms existing rows without changing structure at all — but it can only preserve data the TARGET schema can actually represent, checked directly before any row moves.
2. `Workspace.description` was genuinely missing from the Prisma model — a schema gap, closed with a real, generated migration. `Workspace.ownerId` is real data stored elsewhere (on `WorkspaceMember`) — a field-mapping problem, solved by deriving it inside the script, never by changing `schema.prisma`.
3. No `WorkspaceMember` with `role: 'owner'` exists for that workspace at all; or that member's own `userId` does not exist among the migrated Users.
4. It guarantees a second run leaves an already-migrated row untouched instead of throwing a real unique-constraint error (safe to re-run without duplicating). It does NOT guarantee the row stays synchronized with Mongo if it's later changed independently — `update: {}` never repairs anything.
5. Nest's `ConfigModule.forRoot()` and Prisma's own CLI both load `.env` themselves; a plain `ts-node` script goes through neither mechanism — a real, empirical test confirms it does not load `.env` on its own.
6. Because `id: u.id` / `id: w.id` inserts an explicit id, which the sequence never learns about on its own — a later Prisma-native `create()` (no explicit id) could otherwise collide with an id this script just migrated.
7. Every PRE-EXISTING field the old response actually carried (`id` / `name` / `slug` / `description`) is still present with the same real value; the two NEW fields (`ownerId` / `createdAt`) just need to resolve correctly, since there's nothing old to compare them against; the same real error behavior (status code, message shape) is preserved — not byte-identical JSON.

What you should be able to explain now

The real difference between a schema migration and a data migration, and why a data migration's target schema must be checked, never assumed. Why `Workspace.description` and `Workspace.ownerId` are different kinds of real problem, fixed in different places. Why a real migration script skips and reports rows it cannot safely migrate instead of guessing or crashing. What `upsert()`'s own `update: {}` actually guarantees, and what it does not. Why a standalone script must load its own `.env`, and why Postgres's own autoincrement sequence needs syncing afterward. What "prove old behavior still works" actually requires once the underlying database changes.

Not yet

Decommissioning `MongooseModule` entirely; cutting over any write path (`create` / `update` / `remove`) on `Workspaces`, or any other resource (`WorkspaceMembers` / `Projects` / `Issues`); `Comments`, in any form (confirmed not to exist anywhere in the real application yet); any frontend framework; Prisma's own newer configuration-file/CLI workflow.